acc4cpu

A 4-bit accumulator CPU, from RTL to silicon

A teaching guide to the design and the open-source RTL-to-GDSII flow on the SCL 1.2 µm (C1D) PDK

Silicon No. OS0084 · 3 mm × 3 mm die · 28-pin DIP chip-on-board

Designer: Ayush Jain · Shuttle: NIELIT eChipHub → SCL Mohali MPW

November 2026

How to use this guide

This guide teaches digital chip design end to end using one real, complete project: a tiny CPU that was designed in Verilog, verified, turned into transistors and metal, checked against the foundry's rules, and packaged for fabrication. Every number, file and command in it comes from the actual project folder, so you can open the same files and rerun the same steps.

Read it in order the first time. Each chapter starts with the idea, then shows how this design does it, then lists what went wrong in practice and why. Chapter 16 has exercises.

Contents

1. What we are building and why
2. Background: how a CPU works
3. The acc4 instruction set
4. Architecture: datapath and control
5. The RTL, line by line
6. Verification
7. Synthesis: from Verilog to standard cells
8. Pads, the pad frame and the core wrapper
9. Physical design: floorplan, power, placement, clock tree, routing
10. Physical verification: DRC and LVS
11. Chip assembly: core to pads
12. Signoff: parasitics, timing and post-layout simulation
13. Tapeout packaging
14. Chip art
15. Testing the silicon
16. Exercises
17. Glossary
18. Appendix: files and commands

1. What we are building and why

acc4cpu is a 4-bit accumulator CPU. It has a single working register (the accumulator, ACC), eight instructions, and talks to an external 32-word memory. That memory is not on the chip: an FPGA or a microcontroller board plays the role of memory, which keeps the chip small and lets you single-step the CPU while you watch every signal.

Property	Value
Data width	4 bits (ACC, ALU, data written to memory)
Instruction width	8 bits: 3-bit opcode + 5-bit address
Memory	32 words × 8 bits, external (FPGA / Arduino)
Instructions	LOAD, STORE, ADD, SUB, AND, OR, JMP, JZ (JMP to itself = HALT)
Timing	2 clock cycles per instruction (FETCH, EXEC)
Process	SCL 1.2 µm CMOS, 5 V, 2 metal layers (C1D open PDK)
Die	3 mm × 3 mm, standard SCL pad frame, 28 pins
Logic	~170 standard cells after synthesis (19 flip-flops), ~310 after physical design
Target clock	10 MHz (100 ns period); timing slack ~75 ns

Why so small? Because the point is to learn the complete flow on something you can understand completely. Every gate in this chip can be traced back to a line of Verilog, and every line of Verilog can be traced to a pin on the package.

Key idea: a chip is not "the Verilog". The Verilog is a description. The chip is a set of polygons on about a dozen mask layers (the GDSII file). Everything between the two is the RTL-to-GDSII flow, and most of this guide is about that flow.

2. Background: how a CPU works

2.1 The fetch–execute cycle

A CPU repeats two steps forever. Fetch: read the instruction stored at the address held in the program counter (PC), then add one to the PC. Execute: decode the instruction and do what it says. Real CPUs overlap and pipeline these steps; acc4 does them one after the other, one clock each, which is the easiest version to reason about.

2.2 What an accumulator machine is

An accumulator machine has one data register that is both a source and the destination of every operation. An ADD instruction means ACC ← ACC + memory[address]. Compare this with a register machine (like RISC-V) where an instruction names two sources and a destination. Accumulator machines need very little hardware, which is why early computers (and many 8-bit microcontrollers) used them.

2.3 Registers, combinational logic and the clock

- **Registers (flip-flops)** store state. They change only on the clock's rising edge. acc4 has 19 of them: PC (5), IR (8), ACC (4) and the FSM state (2).

- **Combinational logic** computes outputs from current inputs with no memory: the ALU, the address multiplexer, the zero detector.
- **The clock period** must be longer than the slowest path from one register, through combinational logic, to the next register (the critical path). Static timing analysis (chapter 12) proves this.

3. The acc4 instruction set

Every instruction is one 8-bit word: the top 3 bits are the opcode, the bottom 5 bits are a memory address (0–31).

```
  7   6   5   4   3   2   1   0
+---+---+---+---+---+---+---+---+
|   opcode  |      address      |
+---+---+---+---+---+---+---+---+
```

Opcode	Mnemonic	Meaning	Memory access
000	LOAD a	ACC ← mem[a][3:0]	read
001	STORE a	mem[a] ← ACC	write
010	ADD a	ACC ← ACC + mem[a]	read
011	SUB a	ACC ← ACC − mem[a]	read
100	AND a	ACC ← ACC & mem[a]	read
101	OR a	ACC ← ACC | mem[a]	read
110	JMP a	PC ← a (if a is the JMP's own address: HALT)	none
111	JZ a	if ACC == 0 then PC ← a	none

3.1 Design choices worth noticing

- **ALU select = opcode[1:0].** The four ALU opcodes (010, 011, 100, 101) have low bits 10, 11, 00, 01. Wiring those two bits straight to the ALU means no decoder is needed: ADD=10, SUB=11, AND=00, OR=01.
- **Only the low nibble of data is used.** Memory words are 8 bits (they also hold instructions), but data operations use din[3:0].
- **HALT is free.** Instead of a ninth opcode, a JMP to its own address stops the CPU. This is a classic trick: a self-loop is the natural "stop".
- **Zero flag is combinational.** Z is simply (ACC == 0), computed whenever needed, so there is no flag register to keep consistent.
- **SUB carry is "not-borrow".** SUB is computed as ACC + ~mem + 1; the carry out is 1 when no borrow occurred (the ALU exposes it, the CPU does not use it yet).

3.2 Example program (the one used in every test)

```
addr  word        meaning                ACC after
  0   LOAD  20    ACC = mem[20] = 5      5
  1   ADD   21    ACC = 5 + 3            8
  2   STORE 25    mem[25] = 8            8
  3   SUB   22    ACC = 8 - 8            0   (zero)
  4   JZ    7     taken -> PC = 7        0
  7   LOAD  23    ACC = C                C
  8   AND   24    ACC = C & A            8
  9   OR    21    ACC = 8 | 3            B
 10   STORE 26    mem[26] = B            B
 11   JZ    5     not taken (ACC = B)    B
 12   ADD   27    ACC = B + 7 = 0x12     2   (wraps to 4 bits)
 13   JMP   15    skip address 14        2
 15   STORE 28    mem[28] = 2            2
 16   SUB   21    ACC = 2 - 3            F   (borrow)
 17   STORE 29    mem[29] = F            F
 18   JMP   18    jump to itself -> HALT  F
```

16 instructions × 2 clocks = halt after exactly 32 clocks. Addresses 5, 6 and 14 hold STORE 30 and must never execute; mem[30] is a sentinel that must stay untouched.

4. Architecture: datapath and control

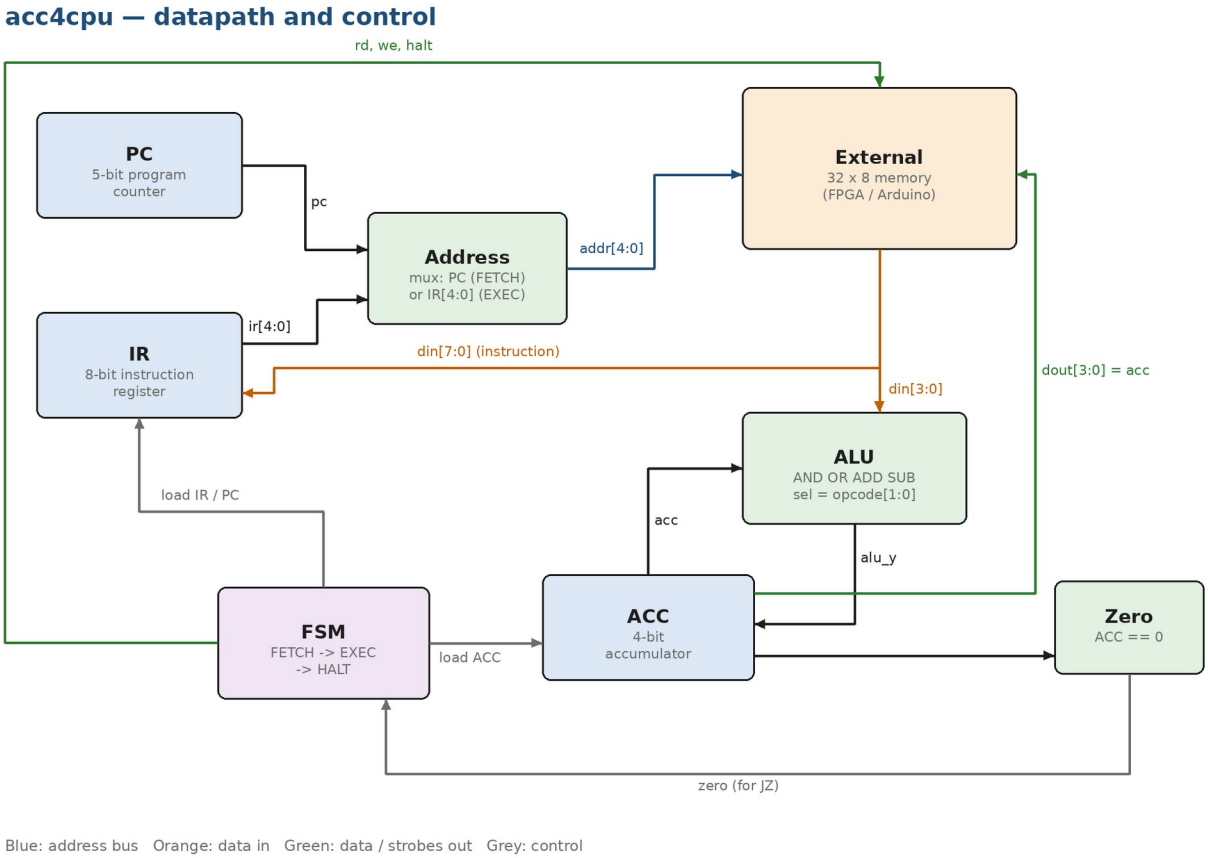


Figure 1 — acc4cpu block diagram. The memory is outside the chip.

4.1 Datapath

- **PC (5 bits)** points at the next instruction; increments in FETCH, loads the target on a taken jump.
- **IR (8 bits)** holds the instruction being executed. Its address field drives the memory address during EXEC.
- **Address mux** chooses PC in FETCH and IR[4:0] in EXEC. This one mux is why instructions and data can share one memory.

- **ALU** AND / OR / ADD / SUB on ACC and din[3:0].
- **ACC (4 bits)** result register; also drives dout to memory for STORE.

4.2 Control: a three-state FSM

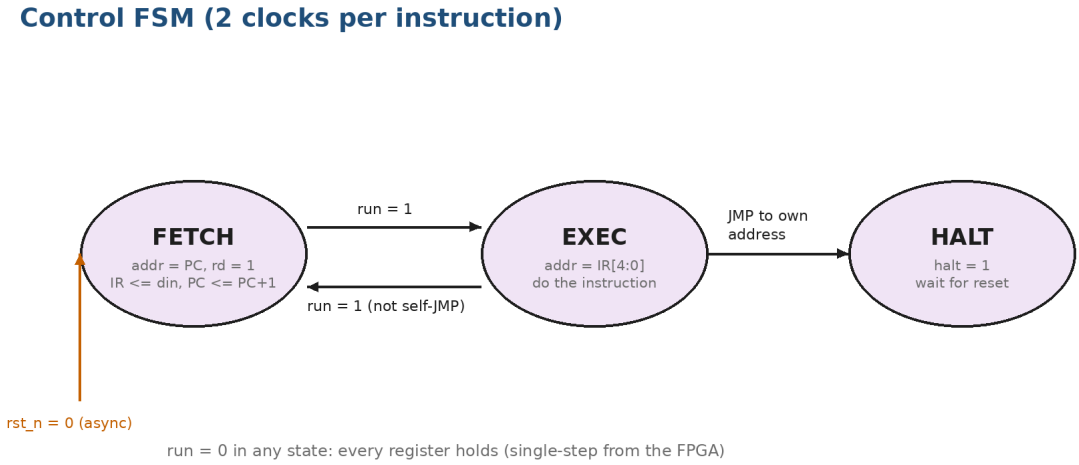


Figure 2 — control state machine.

FETCH puts the PC on the address bus and asserts rd; the memory answers in the same cycle (combinationally), and on the rising edge the IR captures the word and the PC increments. EXEC puts IR[4:0] on the bus and, depending on the opcode, reads an operand, writes ACC (STORE), or changes the PC (JMP/JZ). A JMP whose target equals its own address (PC−1, because PC was already incremented) goes to HALT.

4.3 The bus protocol (what the memory must do)

Signal	Direction	Meaning
addr[4:0]	out	memory address
din[7:0]	in	memory data; must be valid in the same cycle (combinational read)
dout[3:0]	out	data to write (= ACC)
rd	out	1 when the CPU is reading this cycle
we	out	1 when mem[addr] must be written on the next rising clock edge
halt	out	1 once the CPU has halted
clk, rst_n, run	in	clock, asynchronous active-low reset, run (0 = freeze everything)

Why a run pin? With run = 0 every register holds and rd/we are forced low. The external board can therefore advance the CPU one clock at a time and inspect it between clocks: a hardware debugger for free.

5. The RTL, line by line

RTL (register-transfer level) Verilog describes what happens to registers on each clock edge and the combinational logic between them. The synthesizable files are written in plain Verilog-2005

because SCL's tested Yosys (0.9) does not accept SystemVerilog. Every definition has a one-line comment, and every always block has a block comment plus a comment per line.

5.1 The ALU — rtl/acc4_alu.v

```
module acc4_alu (
    input  wire [3:0] a,     // Operand A (accumulator)
    input  wire [3:0] b,     // Operand B (memory data din[3:0])
    input  wire [1:0] sel,   // Operation select (opcode[1:0])
    output reg  [3:0] y,     // 4-bit result
    output reg        cout   // Carry out (ADD) / not-borrow (SUB), 0 for logic ops
);

    localparam SEL_AND = 2'b00;  // Bitwise AND
    localparam SEL_OR  = 2'b01;  // Bitwise OR
    localparam SEL_ADD = 2'b10;  // A + B
    localparam SEL_SUB = 2'b11;  // A - B computed as A + ~B + 1

    wire [4:0] sum_add;          // 5-bit sum for ADD, bit 4 is carry
    wire [4:0] sum_sub;          // 5-bit sum for SUB, bit 4 is not-borrow

    assign sum_add = {1'b0, a} + {1'b0, b};          // Add with carry out
    assign sum_sub = {1'b0, a} + {1'b0, ~b} + 5'd1;  // Two's-complement subtract

    // Combinational result and carry select based on sel
    always @(*) begin
        case (sel)                                   // Decode the operation
            SEL_AND: begin y = a & b;  cout = 1'b0;       end  // AND, no carry
            SEL_OR:  begin y = a | b;  cout = 1'b0;       end  // OR, no carry
            SEL_ADD: begin y = sum_add[3:0]; cout = sum_add[4]; end  // ADD result and carry
            default: begin y = sum_sub[3:0]; cout = sum_sub[4]; end  // SUB result and not-borrow
        endcase                                      // End of operation decode
    end

endmodule
```

Listing 1 — the ALU.

- SUB is built from the same adder as ADD: A − B = A + (~B) + 1 (two's complement). The 5th sum bit is the carry; for SUB it is 1 when A ≥ B (no borrow).
- `always @(*)` with every output assigned in every branch produces pure combinational logic, no latches.
- The `default:` branch catches SUB (and any X during simulation), which is good practice.

5.2 The CPU — rtl/acc4_cpu.v

Ports, parameters and internal signals

```
module acc4_cpu (
    input  wire       clk,    // System clock
    input  wire       rst_n,  // Asynchronous active-low reset
    input  wire       run,    // 1 = execute, 0 = freeze (single-step from FPGA)
    input  wire [7:0] din,    // Data / instruction from external memory
    output reg  [4:0] addr,   // Memory address
    output wire [3:0] dout,   // Data to external memory (accumulator)
    output reg        rd,     // Memory read strobe
    output reg        we,     // Memory write enable
    output wire       halt    // High when CPU has halted
);

    localparam S_FETCH = 2'b00;  // Fetch instruction from mem[pc]
    localparam S_EXEC  = 2'b01;  // Execute instruction held in ir
    localparam S_HALT  = 2'b10;  // Halted until reset

    localparam OP_LOAD  = 3'b000;  // ACC <= mem[a]
    localparam OP_STORE = 3'b001;  // mem[a] <= ACC
    localparam OP_ADD   = 3'b010;  // ACC <= ACC + mem[a]
    localparam OP_SUB   = 3'b011;  // ACC <= ACC - mem[a]
    localparam OP_AND   = 3'b100;  // ACC <= ACC & mem[a]
    localparam OP_OR    = 3'b101;  // ACC <= ACC | mem[a]
    localparam OP_JMP   = 3'b110;  // PC <= a (HALT if a is its own address)
    localparam OP_JZ    = 3'b111;  // PC <= a if ACC == 0

    reg  [1:0] state;      // FSM state register
    reg  [4:0] pc;         // Program counter
    reg  [7:0] ir;         // Instruction register
```

```
    reg  [3:0] acc;        // Accumulator

    wire [2:0] opcode;     // Opcode field of ir
    wire [4:0] opaddr;     // Address field of ir
    wire [4:0] pc_prev;    // Address of the instruction now executing (pc - 1)
    wire       zero;       // Accumulator is zero
    wire [3:0] alu_y;      // ALU result
    wire       alu_cout;   // ALU carry / not-borrow (unused at core level)

    assign opcode  = ir[7:5];         // Split opcode
    assign opaddr  = ir[4:0];         // Split address
    assign pc_prev = pc - 5'd1;       // pc was incremented during FETCH
    assign zero    = (acc == 4'd0);   // Combinational zero flag
    assign dout    = acc;             // Accumulator drives write data
    assign halt    = (state == S_HALT);  // Halt status output
```

Listing 2 — declarations.

- Named `localparam`s for states and opcodes make the case statements readable and avoid magic numbers.
- `pc_prev = pc - 1` exists only to detect the self-jump (HALT): in EXEC the PC has already moved past the JMP.
- `zero` and `halt` are continuous assignments: they are always up to date, with no extra flip-flops.

Combinational bus control

```
    // Combinational bus outputs: address mux and read/write strobes
    always @(*) begin
        addr = pc;                                  // Default: point at program counter
        rd   = 1'b0;                                // Default: no read
        we   = 1'b0;                                // Default: no write
        case (state)                                // Decode bus activity by state
            S_FETCH: rd = run;                      // Read instruction when running
            S_EXEC: begin                           // Execute phase bus activity
                addr = opaddr;                      // Point at operand address
                case (opcode)                       // Decode bus use by opcode
                    OP_LOAD, OP_ADD, OP_SUB,
                    OP_AND,  OP_OR:  rd = run;      // Operand read when running
                    OP_STORE:        we = run;      // Write ACC when running
                    default:         ;              // JMP/JZ use no bus cycle
                endcase                             // End opcode decode
            end                                     // End execute phase
            default: ;                              // HALT: bus idle at pc
        endcase                                     // End state decode
    end
```

Listing 3 — address mux and strobes.

Setting defaults at the top of the block (addr = pc, rd = we = 0) and then overriding them is the safest way to write combinational logic: no path can leave an output unassigned, so synthesis cannot infer a latch. rd and we are ANDed with run, so a frozen CPU never touches memory.

Sequential logic: the FSM and registers

```
    // Sequential FSM, PC, IR and ACC with asynchronous active-low reset
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin                           // Asynchronous reset
            state <= S_FETCH;                       // Start by fetching
            pc    <= 5'd0;                          // Program starts at address 0
            ir    <= 8'd0;                          // Clear instruction register
            acc   <= 4'd0;                          // Clear accumulator
        end else if (run) begin                     // Advance only when run=1
            case (state)                            // FSM state transitions
                S_FETCH: begin                      // Fetch phase
                    ir    <= din;                   // Latch instruction
                    pc    <= pc + 5'd1;             // Point at next instruction
                    state <= S_EXEC;                // Go execute
                end                                 // End fetch phase
                S_EXEC: begin                       // Execute phase
                    state <= S_FETCH;               // Default: fetch next
                    case (opcode)                   // Execute by opcode
                        OP_LOAD:  acc <= din[3:0];  // Load operand
                        OP_STORE: ;                 // Write done by we strobe
                        OP_JMP: begin               // Unconditional jump
                            pc <= opaddr;           // Jump target
                            if (opaddr == pc_prev)  // Jump to itself?
                                state <= S_HALT;    // Then halt
                        end                         // End JMP
                        OP_JZ: if (zero)            // Jump only if ACC == 0
```

```
                                pc <= opaddr;    // Take the branch
                    default:  acc <= alu_y;      // ADD/SUB/AND/OR result
                endcase                          // End opcode execute
            end                                  // End execute phase
            default: state <= S_HALT;            // HALT (and illegal 11) stay halted
        endcase                                  // End FSM
    end                                          // run=0: hold all state
end
```

Listing 4 — the state machine.

- `always @(posedge clk or negedge rst_n)` gives an asynchronous active-low reset. Synthesis maps it to the DFCL11 cell (D flip-flop with clear).
- Non-blocking assignments (<=) are used for every register, so all registers update together at the clock edge.
- `else if (run)` is the freeze: if run is 0 nothing in this block executes, so every register keeps its value.
- Unknown state 2'b11 falls to `default: state <= S_HALT`, a safe fallback.

5.3 The core wrapper and the chip top

Two more modules adapt the CPU to the real pads (chapter 8). acc4_core adds an inverting driver on every output, and acc4_top connects those to OPAD04 output pads, which invert again. The net effect is a true-polarity signal on every pin.

```
    // This contains the instantiation for INVR06
    INVR06 u_drv_addr0 (.IN1(addr[0]), .OUT1(addr_pn[0]));

    // This contains the instantiation for INVR06
    INVR06 u_drv_addr1 (.IN1(addr[1]), .OUT1(addr_pn[1]));
```

Listing 5 — two of the 12 output drivers in acc4_core.v (cells instantiated by name).

6. Verification

Verification is how you convince yourself the design is right before spending a fabrication slot on it. The rule used here: every testbench is self-checking. It compares results with expected values and prints ALL TESTS PASSED or the exact failure. You never judge a waveform by eye.

6.1 The testbenches

Testbench	What it proves
`tb/tb_acc4_alu.v`	All 1,024 combinations of a, b and sel against a reference model (including carry/not-borrow).
`tb/tb_acc4_cpu.v`, test 1	The example program: final memory contents and exactly 32 clocks to HALT.
test 2	HALT really holds: no writes, address frozen.
test 3	run = 0 mid-program (on a pending STORE): nothing changes, nothing is written; the program then finishes identically.
test 4	Asynchronous reset mid-run: address and ACC clear immediately, without a clock edge.

6.2 The same bench at every level

The CPU testbench uses only the top-level ports, so it runs unchanged on the RTL, on the synthesized gate netlist, on the chip top with pad models, and on the routed netlist with real post-layout delays (SDF). One bench, four levels of abstraction. Compile switches select the level: `-DUT_TOP` instantiates acc4_top instead of acc4_cpu, and `-DSDF_FILE` back-annotates delays.

```
# RTL
iverilog -o sim.vvp ../rtl/acc4_alu.v ../rtl/acc4_cpu.v tb_acc4_cpu.v && vvp sim.vvp
# chip top: gate netlist + SCL cell models with timing (max corner)
iverilog -DDUT_TOP -gspecify -Tmax -o gl.vvp $C1D_V ../syn/acc4_core_netlist.v \
         ../rtl/acc4_top.v tb_acc4_cpu.v && vvp gl.vvp
```

6.3 Two bugs the testbench itself had

- **Race at the clock edge.** The first gate-level run failed one check: the bench changed run in the same simulation timestep as the rising edge. In RTL the event order happened to work; with real gate delays it did not. Fix: change inputs and sample outputs in the middle of the low clock phase. Lesson: a testbench must respect setup and hold just like hardware.
- **Too-short settle time.** After dropping run, the bench checked we after 1 ns. With post-layout delays (pad + gates) the signal needed longer. Fix: wait 20 ns, still far inside the 100 ns cycle.

7. Synthesis: from Verilog to standard cells

Synthesis turns RTL into a netlist of standard cells: pre-designed, pre-characterized gates from the foundry's library. The SCL C1D library has 63 cells (inverters, NAND/NOR, AND/OR, XOR, MUX, flip-flops, latches). Each comes in several views: GDS (real layout), LEF (outline and pins, for place and route), Liberty .lib (timing and power), CDL (transistor netlist, for LVS) and Verilog (for simulation).

```
read_liberty -lib /home/ayush-jain/scl/my_dtech/Open_Source_SCL_1.2micron_PDK_digital/
open_source_scl_c1d/open_pdks/sclc1d/digital_c1d/lib/c1d_core_typ.lib

# Read plain Verilog-2005 RTL
read_verilog rtl/acc4_alu.v
read_verilog rtl/acc4_cpu.v
read_verilog rtl/acc4_core.v

# Generic synthesis, flattened; -nofsm keeps the RTL state encoding (reset = 00)
hierarchy -check -top acc4_core
synth -flatten -nofsm -top acc4_core

# Map flip-flops (async-clear DFF -> DFCL11) and logic to the C1D library.
# NND201 is excluded: routers land via1 on its pins too close to its contact/poly (SCL DRC 9.3.1/9.4.1).
dfflibmap -liberty /home/ayush-jain/scl/my_dtech/Open_Source_SCL_1.2micron_PDK_digital/
open_source_scl_c1d/open_pdks/sclc1d/digital_c1d/lib/c1d_core_typ.lib
abc -liberty /home/ayush-jain/scl/my_dtech/Open_Source_SCL_1.2micron_PDK_digital/open_source_scl_c1d/
open_pdks/sclc1d/digital_c1d/lib/c1d_core_typ.lib -dont_use NND201

# Clean up and report
opt_clean -purge
check -assert
stat -liberty /home/ayush-jain/scl/my_dtech/Open_Source_SCL_1.2micron_PDK_digital/open_source_scl_c1d/
open_pdks/sclc1d/digital_c1d/lib/c1d_core_typ.lib

# Gate-level netlist for simulation and OpenLane
write_verilog -noattr syn/acc4_core_netlist.v
```

Listing 6 — syn/synth_acc4_core.ys.

7.1 What each command does

- **read_liberty -lib** teaches Yosys the cells' pin directions without trying to synthesize them. Without it, `check` reported every output as "used but has no driver": a false alarm, because Yosys could not tell which pins of a cell are outputs.
- **synth -flatten -nofsm** generic optimization into abstract gates. -nofsm keeps the state encoding from the RTL; without it Yosys re-encoded the FSM and added a preset flip-flop.
- **dfflibmap** maps flip-flops to library cells. The async-clear register becomes DFCL11.
- **abc -liberty … -dont_use NND201** maps the logic to library gates. NND201 is excluded because during layout the router kept landing vias too close to its contact and gate (a DRC problem found in chapter 10). This is a typical loop: a physical problem fed back into synthesis.

Cell	Count	Function
DFCL11	19	D flip-flop with async clear (all registers)
MX2101	23	2:1 multiplexer
NOR200 / NOR300	19 / 11	2- and 3-input NOR
INVR01	15	inverter
ORND01, ADNR01	13, 9	OR-NAND / AND-NOR compound gates
INVR06	12	strong inverter driving the output pads
others	~45	AND, OR, NAND3, XOR/XNOR …

Total about 167 cells, liberty area 456 (units of the library's unit area).

8. Pads, the pad frame and the core wrapper

A chip talks to the outside world through pads: large metal squares the bonding wires attach to, plus protection (ESD diodes) and drivers. SCL requires one of its standard pad frames. For a 3 mm die the frame has 28 pads: 12 inputs (IPAD00), 12 "outputs" (AIPAD00), 2 VDD and 2 VSS.

8.1 A design decision: the output pads

Reading SCL's netlists showed that AIPAD00 is only ESD diodes and a series resistor: it has no driver. A core cell would have to drive the whole load itself (pad, bond wire, package pin, board trace, FPGA input: roughly 10–15 pF), but the strongest core cells are rated for about 2 pF. Two options:

1. Keep AIPAD00 and accept slow, out-of-spec edges.
2. Replace the 12 AIPAD00 with OPAD04, SCL's real output-driver pad (49.6 pF rating). SCL's frame document allows replacing I/O pads; only the power pads are fixed.

Option 2 was chosen. OPAD04 inverts (PAD = NOT IN1) and its input is a big gate (0.99 pF), more than the 0.81 pF the ordinary cells may drive. So each output first goes through an INVR06 inverter (rated 1.86 pF) in acc4_core. Two inversions cancel, and each stage stays within its rating.

Common misconception: PDDR04 sounds like an output driver but is a pre-driver: it produces P_EN/N_EN gate signals for SCL's bidirectional pads. It cannot drive a pad by itself.

8.2 Pin map

Pad	Signal	Pad	Signal	Pad	Signal
PAD1	addr[0]	PAD9	addr[4]	PAD17	dout[3]
PAD2	clk	PAD10	din[1]	PAD18	din[5]
PAD3	addr[1]	PAD11	dout[0]	PAD19	rd
PAD4	rst_n	PAD12	din[2]	PAD20	din[6]
PAD5	addr[2]	PAD13	dout[1]	PAD21	we
PAD6	run	PAD14	din[3]	PAD22	din[7]
PAD7	addr[3]	PAD15	dout[2]	PAD23	halt
PAD8	din[0]	PAD16	din[4]	PAD24	unused (tie low)

Odd pads are outputs (OPAD04), even pads are inputs (IPAD00). The modified frame was renamed frame3mmx3mm_opad (a changed library structure must not keep SCL's name) and checked on its own: DRC 0, LVS match.

9. Physical design

Physical design turns the netlist into geometry: where every cell sits, how power reaches it, how the clock is distributed, and the exact path of every wire. It ran in OpenROAD (inside the OpenLane 1.0.2 Docker image), following SCL's template scripts, merged into one parameterized script: flow/pnr_core.tcl.

9.1 Floorplan

- Core block 1600 × 1600 µm, placed at (700, 700) inside the 3 mm frame.
- Standard-cell rows are 48.7 µm tall. SCL's template deletes every other row (14 of 28 kept). The empty gaps become routing channels, important with only two metal layers.
- Pins are placed facing their pads (flow/gen_pins.py) so the wires to the pads are short and never cross.

9.2 Power distribution

- A ring of VDD and VSS around the core (metal1 + metal2, 12 µm wide).
- Followpins: 9 µm metal1 rails along every row, which the cells' own power pins abut.
- Two 20 µm metal1 edge stripes (VDD at core y = 608.75, VSS at 998.35), placed in row gaps and running to the block edges. They give the pad frame's VDD and VSS pads something to connect to. OpenROAD's pdngen silently refused to create them, so they are added directly in the database (OpenDB) with via arrays onto the ring.

9.3 Placement and sizing

Global placement spreads cells to minimize wire length at a target density (0.38 here); detailed placement snaps them legally into rows. The resizer (repair_design) then fixes electrical rules: it inserted about 140 buffers (CDRI01) and resized output drivers (the INVR06 drivers became INVR05, which still meets the pad load: 1.49 pF rating vs 0.99 pF).

9.4 Clock tree synthesis

The clock must arrive at all 19 flip-flops at nearly the same time (low skew). CTS builds a buffered tree. SCL's template used INVR01 inverters as clock buffers; this design uses CDRI01, the library's dedicated clock driver, so the clock polarity can never invert by accident.

9.5 Cell abutment (a fix specific to this PDK)

SCL's placement site is only 0.01 µm wide, so cells could land a few hundredths of a micron apart. The p+ implant (boron) of two neighbours then sits a tiny distance apart, a spacing violation. SCL's filler cells only bridge the implant in part of the row, and the smallest filler pushes implant into its neighbour's n-diffusion. Solution: after CTS, any two cells closer than 20 µm are pushed together to abut exactly (the library is designed for abutment), and no fillers are used.

9.6 Routing

TritonRoute connects every net using metal1 (horizontal) and metal2 (vertical). With 2 metals the design is routing-limited, which is why utilization is kept low. The router reported 0 violations of its own rules, but those are only the rules in the LEF abstract; the foundry's full DRC (chapter 10) is stricter.

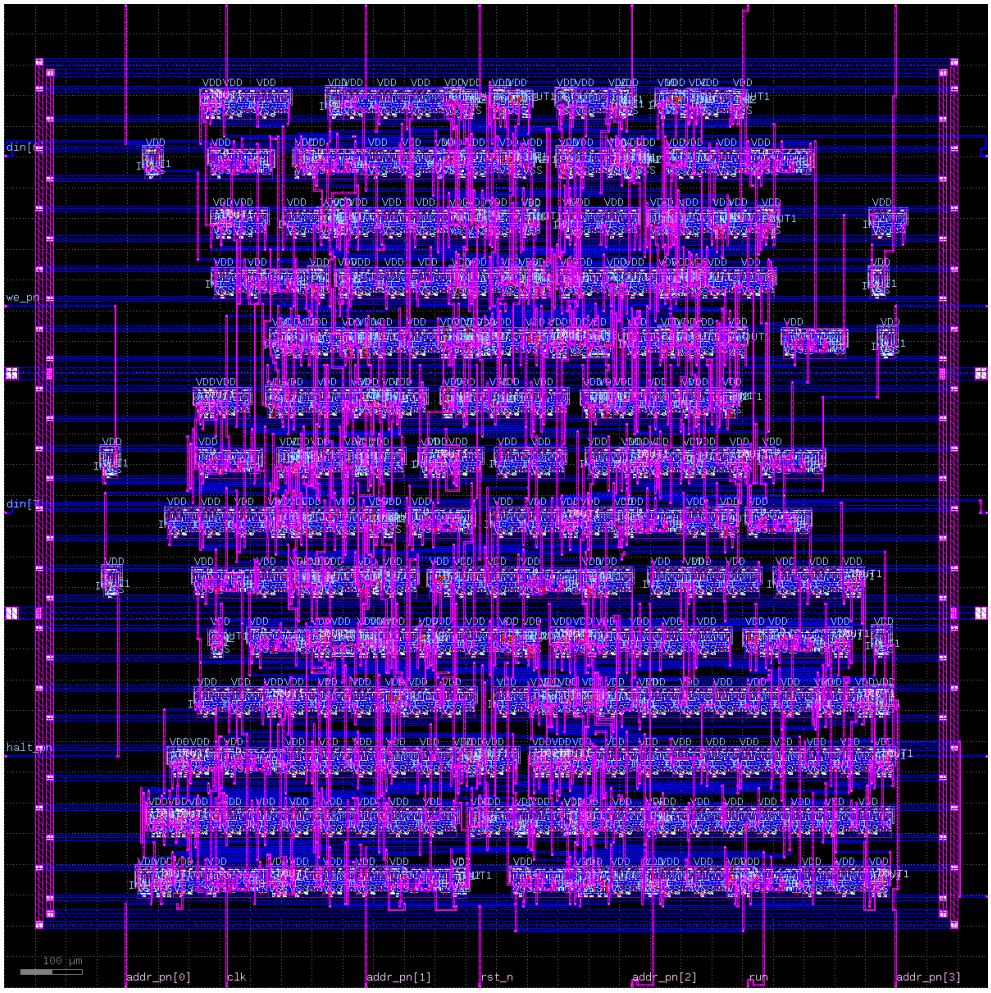


Figure 3 — the routed core (1.6 mm square): 14 cell rows, power ring, edge stripes.

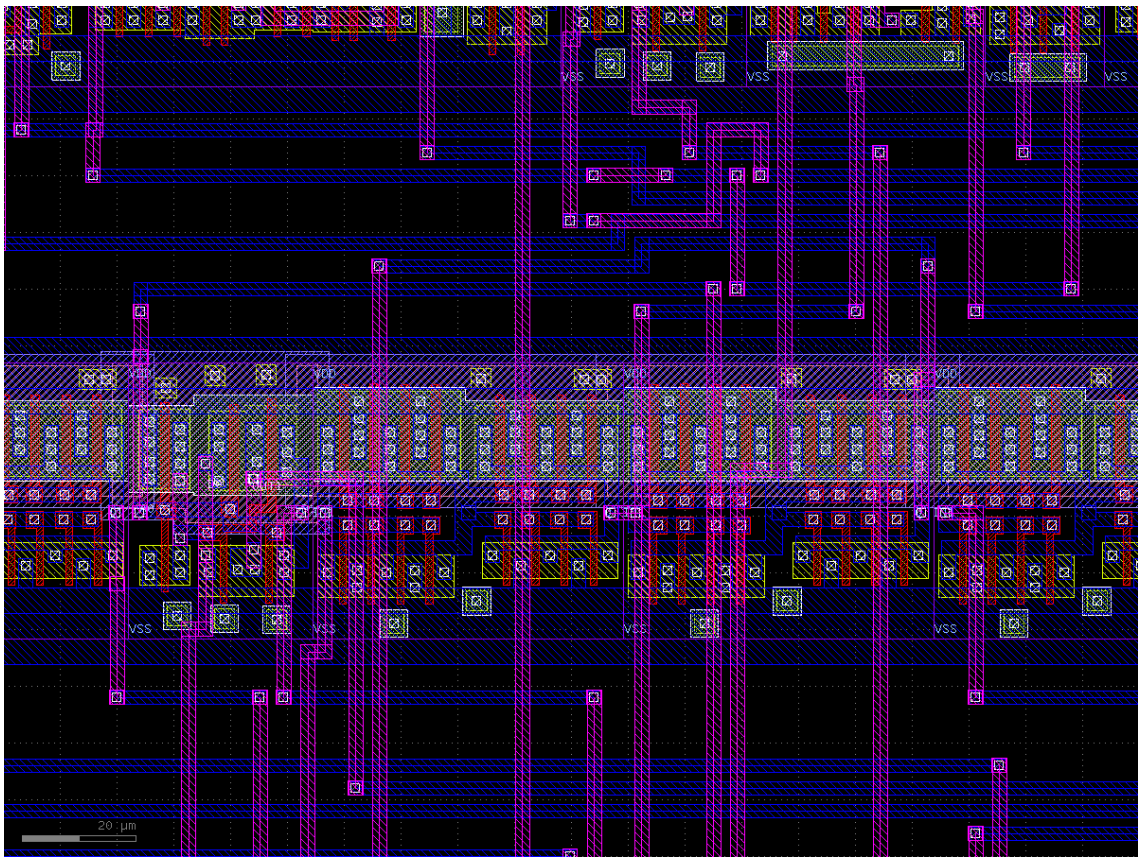


Figure 4 — close-up: a cell row with VDD/VSS rails, metal1 (blue), metal2 (pink), vias.

9.7 Problems met and how they were solved

Symptom	Cause	Fix
`STA-0404 Usage: source`	project path contains spaces	space-free link ~/acc4
edge stripes missing	pdngen drops -nets stripes	create them with OpenDB
`DRT-0416 offgrid pin shape`	SCL's LEF has 1,463 rectangles off the 0.005 µm grid	routing-only LEF copy snapped to grid (snap_lef.py); real GDS untouched
`DPL-0036` placement failed	cells stranded in deleted-row gaps	-max_displacement {500 200}
laptop froze	repair_design with any extra margin runs away in memory	plain repair_design only; every run capped at 6–8 GB, 4 CPUs, 20 min

10. Physical verification: DRC and LVS

10.1 DRC — design rule check

The foundry's rule deck (72 rules for C1D) encodes what the factory can reliably make: minimum widths, spacings, enclosures, overlaps. A design must have zero violations. KLayout runs SCL's deck on the real GDS, with every standard cell's true layout streamed in from SCL's core_c1d.gds.

Rule hit	Cause	Fix
5.7.1 boron spacing (and 5.4.1 / 5.5.1)	tiny gaps between cells; fillers with partial implant	abut cells, no fillers (9.5)
9.3.1 via–contact 1.5 µm, 9.4.1 via–poly 1.2 µm	router puts via1 on NND201 pins near contact/gate	don't use NND201 in synthesis
@GUIDELINE-6 via on diffusion edge	one pin-access via	DRC-driven loop: block that via, re-route; else re-place at another density
10.1.1 metal2 spacing	one via pad 1.5 µm from a wire	same loop

The flow script runs this loop automatically and keeps the first fully clean result (density 0.38 after a few passes).

10.2 LVS — layout versus schematic

LVS extracts transistors and their connections from the polygons and compares them with the intended netlist. A match proves the layout implements the circuit: no missing wires, no shorts, no swapped pins.

- SCL's verilog2spice.py mangled escaped Verilog names (every \u_cpu.* net became one net), so a replacement converter (flow/v2spice.py) was written.
- SCL's std-cell CDL also defines the I/O pads, clashing with the frame's CDL; a filter keeps only core cells.
- Negative test: swapping two pad connections in the reference netlist made LVS fail. A check that cannot fail proves nothing; this one demonstrably can.

11. Chip assembly: core to pads

The routed core and the pad frame are separate layouts. flow/assemble_chip.py places both and draws the wires between them. Each signal leaves its pad on metal2, turns onto its own metal1 track running parallel to the edge, then returns on metal2 into the core pin. Because every net has its own track, no two wires on the same layer cross. The script tries track orders on each side until none clash. Power uses the same scheme with 20 µm straps from the PVDD01/PVSS01 pads to the edge stripes.

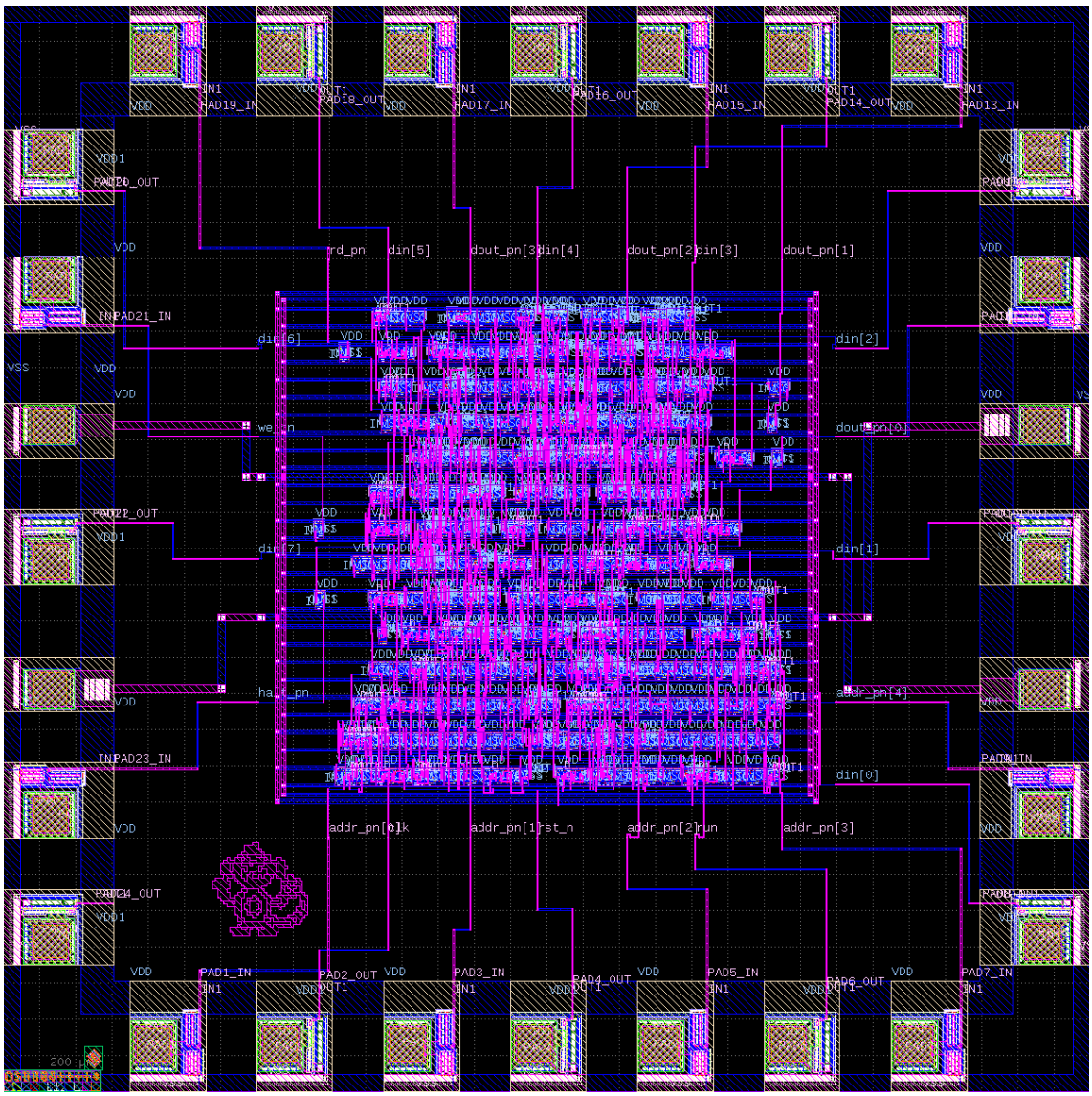


Figure 5 — the complete 3 mm × 3 mm chip: pad ring, channel wiring, core, art (bottom left of core).

Full-chip result: DRC 0 violations, LVS match.

12. Signoff: parasitics, timing and post-layout simulation

12.1 Parasitic extraction

Real wires have resistance and capacitance. OpenRCX computes them from the routed layout with SCL's extraction rules and writes a SPEF file.

12.2 Static timing analysis

OpenSTA checks every path at three process corners (min, typ, max libraries) with a 100 ns clock, 10 ns input/output budgets for the pads and board, and 1 pF on each output.

Corner	Setup slack	Hold slack	Slew / cap / fanout warnings
min	+77.0 ns	+2.0 ns	58 / 38 / 100
typ	+75.6 ns	+3.3 ns	58 / 2 / 64
max	+75.0 ns	+3.7 ns	48 / 0 / 62

Positive slack everywhere: the design meets 10 MHz with a large margin (the critical path is about 25 ns). The warnings are library design-rule limits: signals switching slower than the 2.5 ns characterized maximum (worst 3.15 ns), some nets loaded above a cell's capacitance rating at the min corner, and outputs driving more than the recommended 2–4 inputs. None are on the clock network. They do not stop the chip working at its target speed; they mean delay numbers are extrapolated and the margin absorbs it. Removing them needs a resizer that works with margins (it ran away in this OpenROAD version) or a smaller die.

12.3 Post-layout simulation

The extracted delays are written as SDF and annotated onto the routed netlist; the same testbench runs again. Result: ALL TESTS PASSED at min, typ and max.

13. Tapeout packaging

SCL's submission rules, and how each was met:

Rule	How it was met
GDSII only, top cell = Silicon No. (case-sensitive)	top cell OS0084
Insert SCL's Silicon No. block unmodified	the block's own cell (named OS0084) became the top cell, at its own coordinates in the frame corner under SCL's logo
Origin (0,0), top-right on an even whole µm	(0,0) – (3000, 3000)
0.005 µm grid	all our geometry on grid; only SCL-supplied cells (library, pads, the OS0084 block) contain off-grid points
Clean DRC with SCL's deck	0 violations in 72 rules
Names: SiNo_Design_ddmmyyyy_vN	OS0084_acc4cpu_08112026_v2.gds, _drc.txt, _md5.txt
MD5 checksum	md5sum, saved as _md5.txt

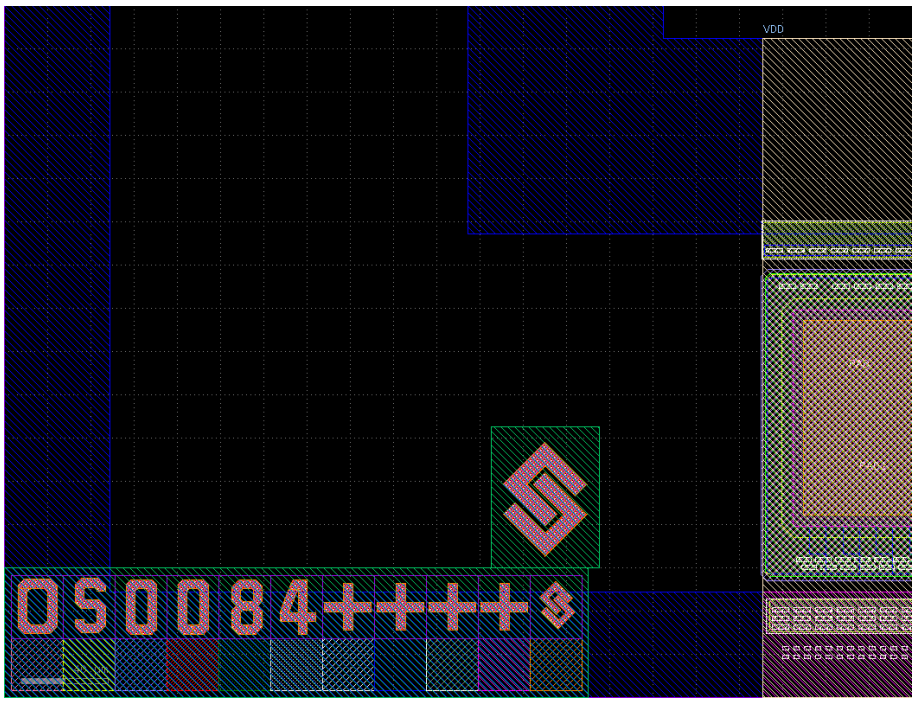


Figure 6 — die corner: SCL's OS0084 Silicon No. block (bottom) under the SCL logo.

14. Chip art

Chip art is a tradition: a small picture drawn in metal that does nothing electrically. This chip carries a monkey taken from the designer's earlier Tiny Tapeout project. The original was 32 × 42 µm on sky130's metal4 with 0.5 µm squares. That is far below this process's minimum width, so flow/make_art.py recovered the original 0.8 µm pixel grid, redrew every pixel as an 8 µm metal2 square (264 × 256 µm), and filled 10 places where pixels touched only at a corner (corner contacts fail DRC). It sits in a free patch next to the core, at least 10 µm from any other shape, and is DRC-clean.

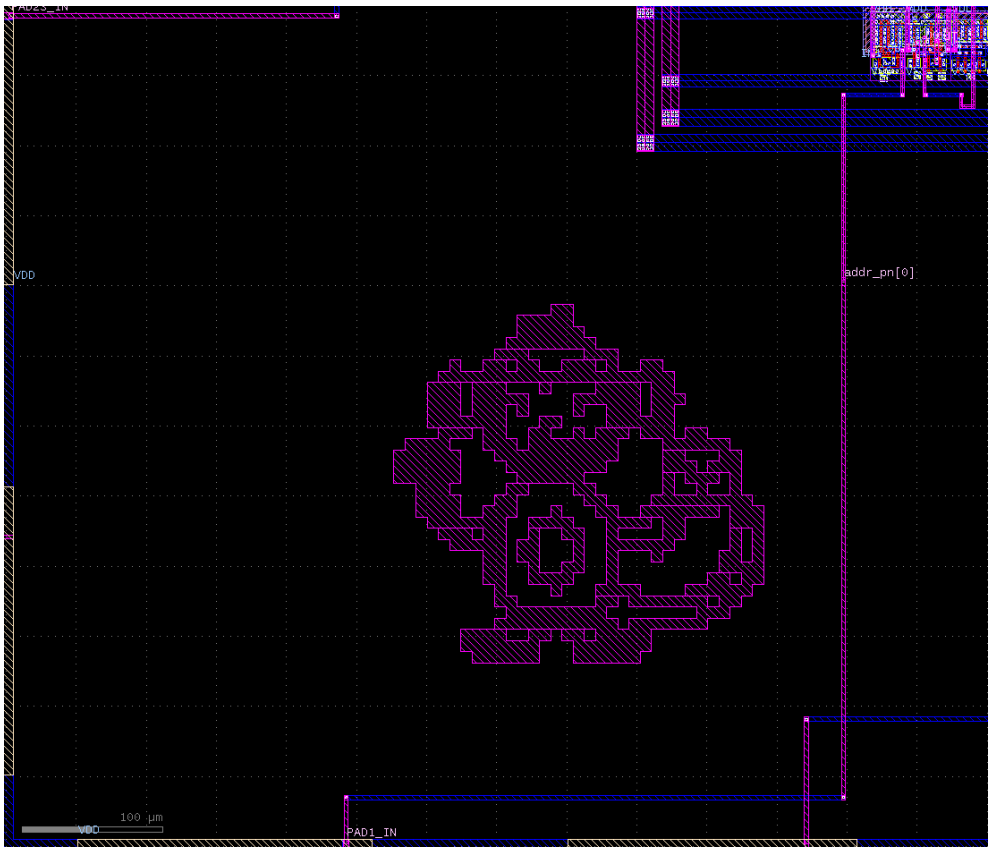


Figure 7 — the monkey in metal2 between the pad wiring and the core.

15. Testing the silicon

1. Inspect the die under a microscope: pad ring, core, monkey, OS0084 block.
2. Power up at 5 V with a 20 mA current limit and decoupling capacitors; hold reset. Expect a few mA.
3. First tester: an Arduino Mega (5 V logic, no level shifting). It plays the 32-word memory: read addr, drive din, pulse clk, capture dout when we = 1.
4. Run the testbench program: expect mem[25]=8, mem[26]=B, mem[28]=2, mem[29]=F, mem[30] untouched, HALT after exactly 32 clocks.
5. Then the exhaustive ALU programs, run/freeze and reset tests.
6. Speed tests on an FPGA with level shifters (74HCT245 into the chip, 74LVC245 out of it), sweeping the clock and the supply voltage.

16. Exercises

1. Add a NOT/XOR instruction. Where would it go in the opcode map, and what must change in the ALU select logic?
2. The ALU produces a carry that the CPU ignores. Add a carry flag and a JC (jump if carry) instruction. How many flip-flops does it cost?
3. Why does HALT compare the jump target with PC−1 and not PC? Draw the timing of FETCH and EXEC to explain.
4. Remove the `else if (run)` freeze and predict which testbench checks fail. Then try it.

5. Calculate the minimum clock period from the STA setup slack. What limits the real maximum frequency on the board?
6. Why did removing every other row help routing with only two metal layers?
7. Explain why a DRC-clean layout can still be wrong, and which check catches it.
8. The resizer inserted ~140 buffers. Look at pnr/acc4_core/acc4_core_rt.v and find one: which net does it drive and why?
9. Design a second chip-art pixel image and put it through make_art.py. Which DRC rules could it violate?

17. Glossary

Term	Meaning
RTL	register-transfer level: Verilog describing registers and the logic between them
Standard cell	a pre-designed gate with fixed height, sitting in rows
Liberty (.lib)	timing/power model of each cell
LEF	abstract of a cell (outline, pins, blockages) for place and route
DEF	placed and routed design in text form
GDSII	the final layout: polygons on mask layers
CDL / SPICE	transistor-level netlist used by LVS
DRC	design rule check: manufacturability rules
LVS	layout versus schematic: the layout implements the netlist
PDN	power distribution network (rings, rails, stripes)
CTS	clock tree synthesis
SPEF	extracted wire resistance/capacitance
SDF	standard delay format: delays for back-annotated simulation
Slack	how much earlier than required a signal arrives (positive = good)
Slew	signal transition time (rise/fall)
PDK	process design kit: everything the foundry supplies to design for its process
MPW	multi-project wafer: many designs share one fabrication run

18. Appendix: files and commands

Path	What it is
`rtl/`	acc4_alu.v, acc4_cpu.v, acc4_core.v, acc4_top.v
`tb/`	tb_acc4_alu.v, tb_acc4_cpu.v
`syn/`	Yosys scripts and netlists
`flow/`	the whole flow as scripts (run_all.sh, finalize.sh, pnr_core.tcl, …)
`layout/`	frame, core, chip and art GDS
`signoff/`	SPEF, SDF, timing reports per corner
`submission/`	the files uploaded to eChipHub
`docs/RUNBOOK.md`	every command and every error, step by step

```
flow/run_all.sh OS0000          # frame -> synth -> P&R -> DRC/LVS -> chip -> signoff
ART=layout/art/acc4_monkey.gds ART_X=576.5 ART_Y=431 \
  flow/finalize.sh OS0084 acc4cpu v2 gds/OS0084.gds    # submission package
```

Full CPU source

```
module acc4_cpu (
    input  wire       clk,    // System clock
    input  wire       rst_n,  // Asynchronous active-low reset
    input  wire       run,    // 1 = execute, 0 = freeze (single-step from FPGA)
    input  wire [7:0] din,    // Data / instruction from external memory
    output reg  [4:0] addr,   // Memory address
    output wire [3:0] dout,   // Data to external memory (accumulator)
    output reg        rd,     // Memory read strobe
    output reg        we,     // Memory write enable
    output wire       halt    // High when CPU has halted
);

    localparam S_FETCH = 2'b00;  // Fetch instruction from mem[pc]
    localparam S_EXEC  = 2'b01;  // Execute instruction held in ir
    localparam S_HALT  = 2'b10;  // Halted until reset

    localparam OP_LOAD  = 3'b000;  // ACC <= mem[a]
    localparam OP_STORE = 3'b001;  // mem[a] <= ACC
    localparam OP_ADD   = 3'b010;  // ACC <= ACC + mem[a]
    localparam OP_SUB   = 3'b011;  // ACC <= ACC - mem[a]
    localparam OP_AND   = 3'b100;  // ACC <= ACC & mem[a]
    localparam OP_OR    = 3'b101;  // ACC <= ACC | mem[a]
    localparam OP_JMP   = 3'b110;  // PC <= a (HALT if a is its own address)
    localparam OP_JZ    = 3'b111;  // PC <= a if ACC == 0

    reg  [1:0] state;      // FSM state register
    reg  [4:0] pc;         // Program counter
    reg  [7:0] ir;         // Instruction register
    reg  [3:0] acc;        // Accumulator

    wire [2:0] opcode;     // Opcode field of ir
    wire [4:0] opaddr;     // Address field of ir
    wire [4:0] pc_prev;    // Address of the instruction now executing (pc - 1)
    wire       zero;       // Accumulator is zero
    wire [3:0] alu_y;      // ALU result
    wire       alu_cout;   // ALU carry / not-borrow (unused at core level)

    assign opcode  = ir[7:5];         // Split opcode
    assign opaddr  = ir[4:0];         // Split address
    assign pc_prev = pc - 5'd1;       // pc was incremented during FETCH
    assign zero    = (acc == 4'd0);   // Combinational zero flag
    assign dout    = acc;             // Accumulator drives write data
    assign halt    = (state == S_HALT);  // Halt status output

    // This contains the instantiation for acc4_alu
    acc4_alu u_alu (
        .a    (acc),
        .b    (din[3:0]),
        .sel  (opcode[1:0]),
```

```
        .y    (alu_y),
        .cout (alu_cout)
    );

    // Combinational bus outputs: address mux and read/write strobes
    always @(*) begin
        addr = pc;                                  // Default: point at program counter
        rd   = 1'b0;                                // Default: no read
        we   = 1'b0;                                // Default: no write
        case (state)                                // Decode bus activity by state
            S_FETCH: rd = run;                      // Read instruction when running
            S_EXEC: begin                           // Execute phase bus activity
                addr = opaddr;                      // Point at operand address
                case (opcode)                       // Decode bus use by opcode
                    OP_LOAD, OP_ADD, OP_SUB,
                    OP_AND,  OP_OR:  rd = run;      // Operand read when running
                    OP_STORE:        we = run;      // Write ACC when running
                    default:         ;              // JMP/JZ use no bus cycle
                endcase                             // End opcode decode
            end                                     // End execute phase
            default: ;                              // HALT: bus idle at pc
        endcase                                     // End state decode
    end

    // Sequential FSM, PC, IR and ACC with asynchronous active-low reset
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin                           // Asynchronous reset
            state <= S_FETCH;                       // Start by fetching
            pc    <= 5'd0;                          // Program starts at address 0
            ir    <= 8'd0;                          // Clear instruction register
            acc   <= 4'd0;                          // Clear accumulator
        end else if (run) begin                     // Advance only when run=1
            case (state)                            // FSM state transitions
                S_FETCH: begin                      // Fetch phase
                    ir    <= din;                   // Latch instruction
                    pc    <= pc + 5'd1;             // Point at next instruction
                    state <= S_EXEC;                // Go execute
                end                                 // End fetch phase
                S_EXEC: begin                       // Execute phase
                    state <= S_FETCH;               // Default: fetch next
                    case (opcode)                   // Execute by opcode
                        OP_LOAD:  acc <= din[3:0];  // Load operand
                        OP_STORE: ;                 // Write done by we strobe
                        OP_JMP: begin               // Unconditional jump
                            pc <= opaddr;           // Jump target
                            if (opaddr == pc_prev)  // Jump to itself?
                                state <= S_HALT;    // Then halt
                        end                         // End JMP
                        OP_JZ: if (zero)            // Jump only if ACC == 0
                                    pc <= opaddr;   // Take the branch
                        default:  acc <= alu_y;     // ADD/SUB/AND/OR result
                    endcase                         // End opcode execute
                end                                 // End execute phase
                default: state <= S_HALT;           // HALT (and illegal 11) stay halted
            endcase                                 // End FSM
        end                                         // run=0: hold all state
    end

endmodule
```

Listing 7 — rtl/acc4_cpu.v.